

四张 NPU 的 I/O stall：什么能算准，什么只能预测？

从单层界限、FIFO 长突发，到多层递推和 CIR 预算

2026-09-06 · 第一版；模型推导与实验结论分开

Contents

1 先把问题分成三件事	1
2 单位、硬件和预取假设	2
2.1 所有符号对应什么	2
2.2 计算固定，不代表剩余预算一直固定	2
3 当前层：不需要知道队列内部顺序的判断	3
3.1 适用位置：目标这一层尚未提交任何 I/O	3
3.2 当前项目是逐条发射，不能默认原子入队	3
3.3 可直接运行的例子	3
4 更强的结论：什么长突发使自然错开也不能消除 stall？	4
4.1 一个相位无关的充分条件	4
4.2 逐条提交的大批次，也可以计算保守的 H	4
4.3 有限 8 层和无限重复，不能混为一谈	5
5 为什么必须做多层递推？	5
5.1 FIFO 的 max-plus 递推	5
5.2 同一个队列数量，可以有相反的后续结果	6
6 Python 输入与结果：不把猜测当遥测	6
6.1 NPUState 对应真实什么信息？	6
6.2 完整使用例子	6
6.3 为什么结果叫 conditional_prediction？	7
7 从 stall 预测到 Path/CIR 规划	7
7.1 平均负载接近 1，仍然不是每层及时的充分条件	7
7.2 一个能指导 CIR 的简单预算	7
8 验证方法与局限	8

1 先把问题分成三件事

本说明针对四张 NPU 各自独立处理请求，不是一个请求在四张卡上做张量并行。

1. **当前这一层是否会等数据？**可以在明确条件下给出严格界限。
2. **新请求会不会在后续层拖慢自己或别人？**需要递推计算、预取和排队的反馈；当前排队数量不能单独决定答案。
3. **什么情况下自然错开也不能消除等待？**长突发可以形成一段不可越过的 FIFO 工作量。第 4 节给出一个不依赖具体相位的充分条件。

Python 文件 `npu_stall_predictor.py` 同时提供单层筛查、长突发证明、多层条件预测和加入请求前后的差分。它不导入项目仿真器，不把读取未来 trace 当作预测。

实际策略实验见同目录上一级的 `stall_policy_experiment_report.pdf`。**公式的严格界限、条件预测与实测策略收益是三个不同的结论。**

2 单位、硬件和预取假设

2.1 所有符号对应什么

符号	含义与单位
b	一条 I/O 的字节数，固定 $176 \times 1024 = 180224$ byte
K_i	NPU i 某层需要读取的完整 KV block 数，也是 I/O 数
B	SSD 总带宽，byte/s；代码参数是 GiB/s，须乘 2^{30}
C_i	当前层计算总时长，ms；各 NPU 可以不同
t	当前时刻，ms；函数中快照时刻统一记为 0
D_i	下一层数据必须到齐的时刻，ms
Q	Path0 尚未完成 SSD 服务的 I/O 数， 包含在途、不含目标新层
s	一条 I/O 的 SSD 服务时长，ms
ℓ	一条 I/O 的 SSD→HBM 接收时长，ms

一个 KV block 是 128 token 的每层 KV；项目格式每 token 每层 1408 byte，因此 $128 \times 1408 = 176$ KiB。不要再乘一次 128。

这里每条 I/O 都是完整块。旧教程部分请求存在不足 176 KiB 的最后一条 I/O；新的完整块实验与旧 trace 不是字节级相同输入，不能静默互换。

在 SSD 为 40 GiB/s、每卡独立接收链路为 50 GiB/s 时：

$$s = \frac{176 \times 1024}{40 \times 2^{30}} \times 1000 = 0.0041961669921875 \text{ ms},$$

$$\ell = \frac{176 \times 1024}{50 \times 2^{30}} \times 1000 = 0.00335693359375 \text{ ms}.$$

单 SSU、完整等大小 I/O、合法历史状态下，SSD 相邻完成至少相隔 s ，而一条接收只要 $\ell < s$ ，所以每卡接收链路不会积压。但最后一条仍要加上接收尾延迟。若链路比盘慢、多盘同时汇入、I/O 大小不一或有其他链路流量，必须扩展模型，不能继续直接加一个固定尾。

2.2 计算固定，不代表剩余预算一直固定

当前层在 $S_{i,l}$ 开始、计算 C_i ，则下一层需数时刻为：

$$D_{i,l+1} = S_{i,l} + C_i.$$

在时刻 t ，剩余预算是 $D_{i,l+1} - t$ 。把读取拖晚不能把 deadline 一起往后重置。函数中 `deadline_ms` 是**快照之后还能等多久**，不是让新 I/O 从实际下发时重新获得一个完整计算周期。

项目在当前层开始计算时激活下一层预取。跨请求预取仅在当前请求最后一层开始计算时，针对**当时已经到达**的下一请求触发。若新请求在这个时刻之后才到达，不会补触发这次预取。

3 当前层：不需要知道队列内部顺序的判断

3.1 适用位置：目标这一层尚未提交任何 I/O

当所有工作使用同一个严格 FIFO Path0 时，已有 Q 条 I/O 全都在这个新层之前。因此，判断这个新层的前方总工作量，**不需要知道旧队列内部哪个 NPU 排在谁前面**。

设在途 I/O 的剩余量为 $r \in [0, b]$ 。如果 $Q > 0$ ，前方工作是 $(Q - 1)b + r$ ；如果 $Q = 0$ ，前方工作为 0。

目标有 $K > 0$ 条 I/O。假设本批能立即完整排到队尾：

$$T(r) = \frac{(Q - 1)b + r + Kb}{B} \times 1000 + \ell \quad (Q > 0).$$

若拿不到 r ，仍可得到：

$$\begin{aligned} T_{\min} &= (\max(Q - 1, 0) + K)s + \ell, \\ T_{\max} &= (Q + K)s + \ell. \end{aligned}$$

$Q = 0$ 时二者相同； $K = 0$ 时数据已经无需读取，返回 0，不等待旧队列、不增加接收尾。

判定方法：

- $T_{\min} > D$ ：即使最乐观也来不及，must_stall。
- 已明确允许原子完整入队，且 $T_{\max} \leq D$ ：meets_deadline。
- 其余情况：unknown。

unknown 不是已经发生 stall，而是当前信息不足以保证及时。

3.2 当前项目是逐条发射，不能默认原子入队

客户端每张 NPU 每 0.1 微秒发一条 I/O。目标本批尚未发完时，其他 NPU 新产生的命令可能排在它的后续 I/O 前面。

此时上面的 $T_{\min}$ 仍是乐观下界：已经排在队列里的工作不能越过。但在不知道之后会插入多少工作时，**不能把 $T_{\max}$ 当成保证上界**。所以函数默认 atomic_enqueue=False，上界返回 None。

如果目标层已有部分 I/O 入队，整个 Path 的 Q 可能包含目标自己的工作，以及排在目标之后的工作。此时不能把 $Q + K$ 机械相加；请使用第 5 节的状态输入，而不要调用这个单层筛查接口。

3.3 可直接运行的例子

```
from npu_stall_predictor import screen_current_layer
```

```
result = screen_current_layer(
    kv_blocks=10,
    deadline_ms=0.5,
    path_outstanding_io=100,
    bandwidth_gib_s=40,
    link_gib_s=50,
)
```

这里 $Q = 100$ **包含**在途 I/O，剩余量未知。乐观完成时间为 0.4607391357421875 ms，默认返回 unknown：本批还可能在逐条提交过程中遭遇其他流。

如果明确设置 atomic_enqueue=True，上界为 0.464935302734375 ms，这次可以保证赶上 0.5 ms。若 deadline 改为 0.45 ms，连下界也超时，可以判 must_stall。

旧函数 `glm_path0_deadline.py` 的 `queued_io_count` 不含在途 I/O, 这里的 `path_outstanding_io` 包含在途；两个参数不能不换位就直接对照。

4 更强的结论：什么长突发使自然错开也不能消除 stall？

4.1 一个相位无关的充分条件

假设在时刻 a ，长流的一批 I/O 已经全部排进 FIFO，盘前方还有至少 H ms 的服务工作。

考虑一张短计算 NPU：每层计算 C ms，下一层自己需要 $w = Ks$ ms 的 SSD 服务。它当前在计算，且还有足够后续层，能够再次发起预取。

先反设它接下来完全不 stall。那么下一次计算开始、同时发起再下一层读取的时刻 r 必须满足：

$$r \leq a + C.$$

这次读取的 deadline 因而不晚于：

$$D = r + C \leq a + 2C.$$

但这次新读取不可能越过已经排好的 H ，所以数据到齐最早也要：

$$R \geq a + H + w + \ell.$$

因此，如果：

$$\boxed{H + w + \ell > 2C},$$

那么“接下来完全不 stall”的假设矛盾。**不管短流原本处在什么相位，只要它接下来仍需按此规则读取，就必须出现一次 stall。**如果它在下一次释放之前就已经等数据，同样已经说明无法保持无 stall。

这不是必要条件。不满足不等式，只能说本证明不能判定，不能反推出一定没事。

4.2 逐条提交的大批次，也可以计算保守的 H

长流一批共有 K_b 条 I/O，第一条到最后一条的提交跨度为 J ms。整个批次的服务工作是 $K_b s$ ；提交期间 SSD 最多消耗 J ms 工作。因此，在最后一批提交完时：

$$H \geq \max(0, K_b s - J).$$

例如完整块重构输入中的长批 $K_b = 1534$ ，在没有额外客户端发射间隙时：

$$J = (1534 - 1) \times 0.0001 = 0.1533 \text{ ms},$$

$$H_{\min} = 6.283620166015625 \text{ ms}.$$

短流 $K = 7$ 、 $C = 0.582149491632$ ms 时， $H_{\min} + 7s + \ell$ 远大于 $2C$ 。这个证明解释的是：**长流在很短时间内把超过短流几个计算周期的工作排在前面；仅靠层间自然错位不可能持续避免它。**

对应函数：

```

from npu_stall_predictor import fifo_burst_certificate

certificate = fifo_burst_certificate(
    burst_kv_blocks=1534, burst_submit_ms=0.1533,
    victim_kv_blocks=7, victim_compute_ms=0.582149491632,
)

```

certified_excess_lower_ms 是接下来两次计算切换的**累计 stall 下界**，不一定落在同一层，也不是整段请求的总 stall 预测值。第一层若已经迟到，会推迟第二次读取与 deadline，不能把这个差额全算到第二层头上。

具体地，当前计算还剩 $x \leq C$ ，两次切换的 stall 分别为 W_1, W_2 。第二次预取在 $a + x + W_1$ 释放，到齐仍不早于 $a + H + w + \ell$ ，因此：

$$W_1 + W_2 \geq H + w + \ell - x - C \geq H + w + \ell - 2C.$$

这个推导也解释了为什么仅证明“至少有一次 stall”，不能直接断言某一层独自等待了全部差额。

4.3 有限 8 层和无限重复，不能混为一谈

证明需要短流还有足够层，在当前计算之后继续发起一次下一层预取；接近请求末尾时，可能已经没有这次未来读取。只有后续同类请求持续到达、跨请求预取条件成立，才能讨论长期反复触发。

要声称某个周期永远重复，需要完整相位状态在一个周期后重现，不能只看到两个峰就断言。实际报告会区分“8 层内复发”“测量窗口内持续出现”和“已证明的无限周期”。

5 为什么必须做多层递推？

5.1 FIFO 的 max-plus 递推

按实际入队顺序编号 I/O。第 k 条在 A_k 时刻提交，SSD 完成为：

$$F_k = \max(A_k, F_{k-1}) + s.$$

一般情况下，接收链路还需递推：

$$G_k = \max(F_k, G_{\text{prev, same NPU}}) + \ell.$$

本说明的等长、单 SSD、50 GiB/s 独立链路条件下，化简为 $G_k = F_k + \ell$ 。

一层数据到齐及计算开始为：

$$R_{i,l} = \max_{k \in (i,l)} G_k,$$

$$S_{i,l} = \max(S_{i,l-1} + C_i, R_{i,l}).$$

其 stall 为：

$$W_{i,l} = \max(0, R_{i,l} - (S_{i,l-1} + C_i)).$$

关键反馈是： $S_{i,l}$ 又触发第 $l + 1$ 层的 I/O 提交。所以输入时机、排队、stall、下一轮时机互相决定，不能把后续各层发起时刻一直写成初始时刻加 lC 。

5.2 同一个队列数量，可以有相反的后续结果

教学例子里，当前 $Q = 0$ ，新卡每层计算 4 秒、读 1 秒。它当前的预取及时，下一次在第 4 秒发起，deadline 为第 8 秒。

若另一卡第 3 秒释放需要服务 6 秒的读取，新卡就要等到第 10 秒；若另一卡到第 10 秒才释放，新卡第 5 秒就拿到。当前 Q 、新请求参数、另一卡的读取量都相同，差别只在另一卡**当前计算还剩多久**。

因此，需要各卡的计算进度，而不只是盘的压力计数。

6 Python 输入与结果：不把猜测当遥测

6.1 NPUState 对应真实什么信息？

每张卡提供一个 NPUState，四张卡组成长度为 4 的列表：

字段	含义
request	当前请求的 ID、每层完整块数、计算时间、层数
next_layer	下一层尚未开始计算的层号；当前请求层已全部开始时可等于层数
compute_end_in_ms	当前计算将在快照后多久结束；已结束可为 0 或负数
queued_io	下一待计算层仍在 SSD 的 I/O 数，包含它可能拥有的在途命令
unissued_io	该层已激活但尚未发出的 I/O 数；None 表示尚未激活
ready_in_ms	SSD 已读完但 HBM 尚未到齐时的剩余到齐时间
next_issue_in_ms	客户端下次允许发射的剩余时间
waiting_requests	已到达但待接纳的请求，按该 NPU 自己的接纳顺序排列

卡自身的接纳顺序不等于 SSU 内的 FIFO 顺序。前者客户端掌握；后者默认未知。每卡 `queued_io` 的总和应与 Path0 压力对账，注意 SSD→HBM 在途不再计入 Path0 压力。

若这些计数不是同一时刻采集、只有过期值或只能看到所有权不明的总量，第一版多层函数不能返回有保证结果。反馈延迟需要在后续实验中加入误差和开销。

6.2 完整使用例子

```
from npu_stall_predictor import (
    Request, NPUState, predict_request_impact,
)

npus = [
    NPUState(),
    NPUState(
        request=Request(10, kv_blocks=64, compute_ms=0.5),
        next_layer=1,           # 当前正在计算 L0
        compute_end_in_ms=0.02, # 还需计算 0.02 ms
        unissued_io=0,         # L1 已激活且全部提交
        queued_io=0, ready_in_ms=0, # L1 已在 HBM
    ),
    NPUState(), NPUState(),
]
```

```
new = Request(99, kv_blocks=128, compute_ms=0.8)
report = predict_request_impact(npus, new, target_npu=0)
```

`request_id` 只是全局唯一身份编号，不是 NPU 编号、也不是请求大小。结果通过 (`request_id`, `layer`) 对齐同一层，不依赖请求 ID 的数字结构。

candidate_layers 给出新请求每层的 deadline、预测到齐、计算开始/结束和 stall。existing_request_completion_delay_ms 给出已有请求完成时间相对于“不加入”的变化。负值意味着在这个条件场景里因为相位改变而提前，不应偷偷裁成 0。

stall_ms 对快照前已经等待的当前层可包含历史欠账；future_stall_ms 只计快照之后的等待。Layer0 会单独标记；首次读料不能当作自然错开失败的证据，但完整请求延迟仍须包含它。

6.3 为什么结果叫 conditional_prediction？

函数只用各 NPU 的排队数量，按轮询或分组方式重建一种**可能的**初始 FIFO 顺序。未来同时发射的 NPU 使用声明的固定顺序，而项目模拟器使用带种子的随机顺序。正在服务的剩余量未知时，默认按完整一条计算。

这些都是场景假设，不是现实队列的观测，也不保证给出整个闭环最坏值。impact_order_sensitivity 会尝试 24 种顺序；其 min/max 只是敏感性样本范围，**不是遍历所有 I/O 排列得出的数学上下界**。

第一版不直接支持有限 PIR 或 256 Path 的精确虚拟完成标签预测。在线策略不能使用隐藏的 FIFO/virtual-finish 状态来冒充可部署收益。策略实验使用当前公开统计和请求元信息，预测误差另行核验。

7 从 stall 预测到 Path/CIR 规划

7.1 平均负载接近 1，仍然不是每层及时的充分条件

固定同类流、不考虑 stall 的名义需求：

$$\rho = \sum_{i=0}^3 \frac{K_i b}{B(C_i/1000)}.$$

$\rho > 1$ 时，持续运行的四流不可能全部无 stall。 $\rho \leq 1$ 只排除总平均容量超载，不能排除第 4 节的 FIFO 长突发阻塞。单卡的 $K_i s + \ell > C_i$ 也会使它即便独占 SSD 仍然来不及。

新请求可分配 NPU 后，每卡任务组合会改变；不能把原固定四流的 ρ 当成策略 B 每个时刻的负载。比较必须保持请求集和到达时间相同，同时报告完整请求集的完成时间，防止只在一秒内换成更容易执行的任务。

7.2 一个能指导 CIR 的简单预算

对于由客户端控制归属的 Path，若当前前方工作为 q 条、目标还有 K 条，预算为 D ms，先给不可忽略的延迟留出 L ms。流体模型所需服务率是：

$$r_{\text{need}} = \frac{(q + K)b}{(D - L)/1000} \text{ byte/s}.$$

函数 required_rate_gib_s 输出对应 GiB/s。若 $D \leq L$ 且还有工作，返回无穷，表示该预算无法满足。

只有当实际调度器拥有已证明的服务曲线 $r(t - L)^+$ 时，这才是可作保证的充分预算。**当前 CIR 的长期服务份额，不等于从现在起每个微秒都以 CIR 传输**。非抢占命令和已有虚拟完成历史会影响短期时序，故实验策略首先把它当规划估计。

256 条 Path 共用一个物理后端，不是带宽乘 256。所有 Path 的 CIR 总和仍不超过 40 GiB/s；CIR=0 不一定没有服务，它仍可能借用剩余服务机会。当前项目拒绝有限 PIR，因此本轮 PIR 保持不限。

初始策略 A 使用 256 条可选 Path 中的四条独立 Path，再按每卡 V/C 需求预留 CIR。均分带宽是消融对照，不假设四张卡需要相同带宽。策略 B 在到达时再按已知未完成工作选择 NPU，不改变到达时间，不迁移已运行请求。

8 验证方法与局限

```
python -m unittest -v test_npu_stall_predictor
python npu_stall_predictor.py
```

25 项独立单元测试核验公式边界、接收尾、未知顺序、部分提交、跨请求预取和差分。validate_stall_predictor.py 已与原项目仿真器比较真实逐层结果；不是把独立函数自己生成的数据作为验证真值。

实测对照	最大绝对误差
单 NPU 冷启动，8 层	0
四 NPU 冷启动，原生随机提交顺序，3 个 seed	4.196-8.392 微秒
已有队列的动态快照：数据到齐	0.461121 ms
同一动态快照：计算开始、快照后 stall	0.079269 ms
候选造成既有请求完成延迟：预测增量误差	0.003738 ms

动态验证在仿真第 1.25 ms 捕获真实状态：Path0 有 213 条 SSD 未完成 I/O，其中 NPU1 已入队 46 条、尚未提交 82 条；有的 NPU 已在等待。新候选实际使另一请求延后约 1.725-1.730 ms。这里两次采样未出现 stall 分类误报/漏报，**不构成准确率保证**。动态队列误差明显比空队列大，也证明不能把未知顺序忽略掉。

验证适配器额外记录 SSU 按请求/层的完成计数和已观察完成时间，用来区分 SSD outstanding 与 HBM 尾；当前只有 Path 总计数的接口不能直接提供这些信息。它没有读取 FIFO 顺序、正在服务 I/O 的身份、剩余量或未来事件。

实验中另实现 deadline_qos_controller.py：使用客户端能维护的未完成块数、当前计算结束时刻，在每层开始/到齐时重新选择 CIR 优先方向。它不调用上述完整 Path0 预测器作为 256 Path 的精确预测器，而是使用 deadline 算术指导实际调度；CIR 倾向 EDF 不等于底层已实现 EDF。策略收益与反例见实验报告。

原仿真使用恒定计算和简化 SSD 数据面。真实部署还需测：状态采集延迟、CIR 写入延迟、决策 CPU 时间、真实服务曲线、计算波动、SSD 内部并行/GC，以及 176 KiB 命令是否被实际驱动拆分。实验中这些没有建模的成本会明确列出，不宣称仿真收益等同硬件收益。

本项目证据入口：sim.py 中的 PathQueue、DiskIOScheduler；continuous_batch_sim.py 中的 _handle_compute_schedule、_start_cross_request_layer0_prefetch 和接收链路处理；continuous_prefill_client.py 中的发射间隔与 baseline/layer_once 配置。公式为上述限定模型的直接推导。